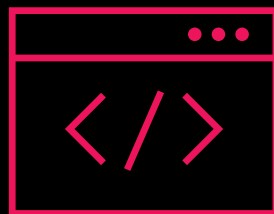


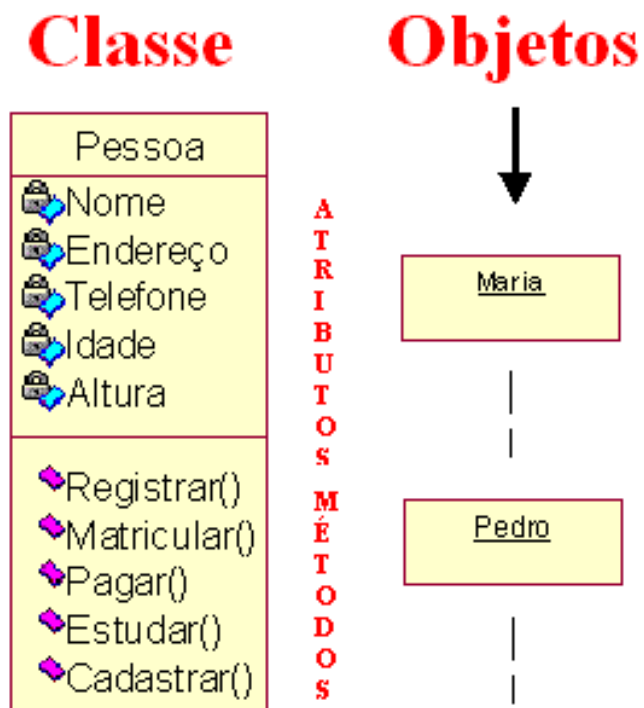
ORIENTAÇÃO A OBJETOS

Pensamento Computacional e Automação com Python

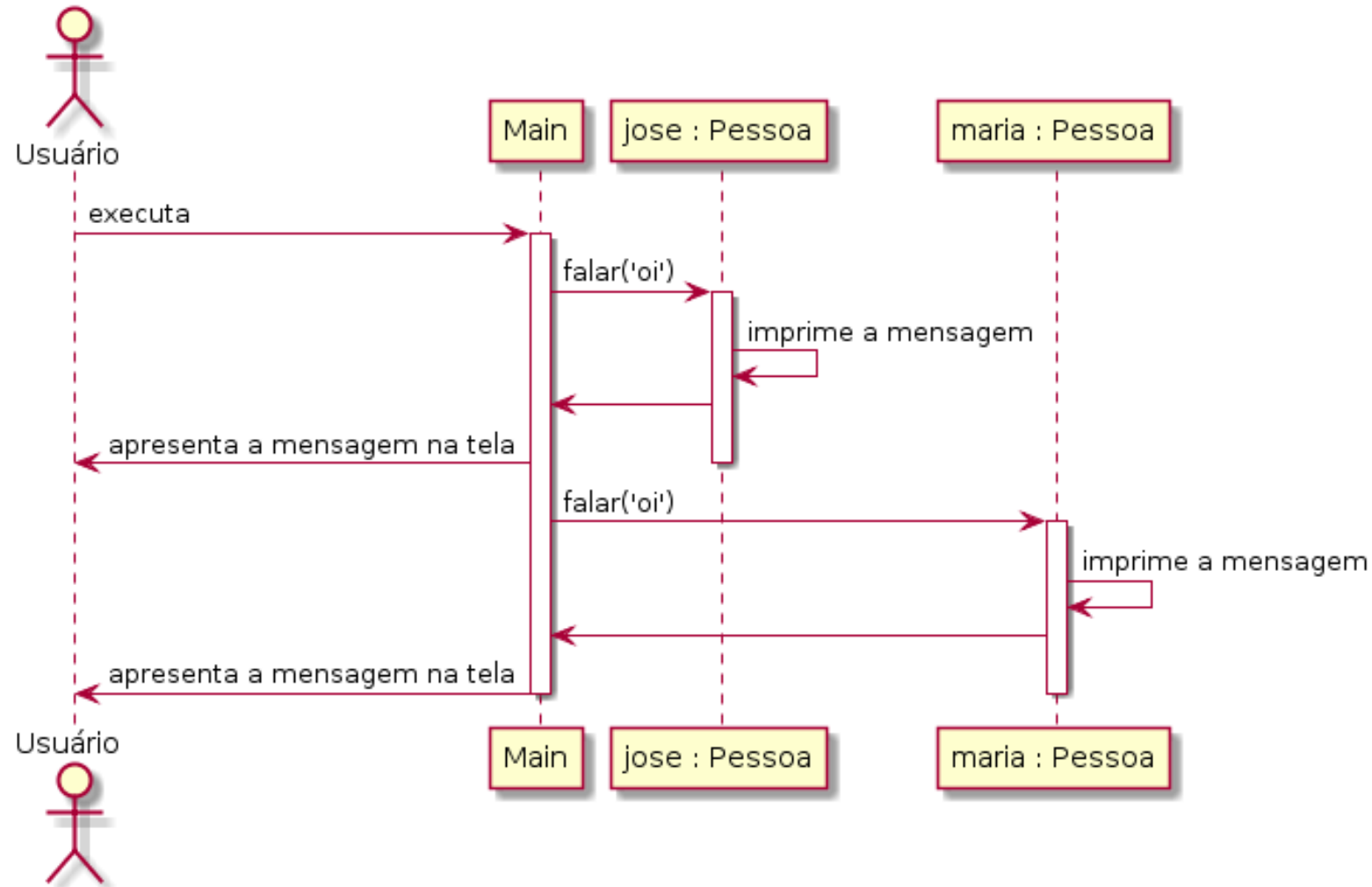


Orientação a Objetos: o básico com Aluno e Disciplina

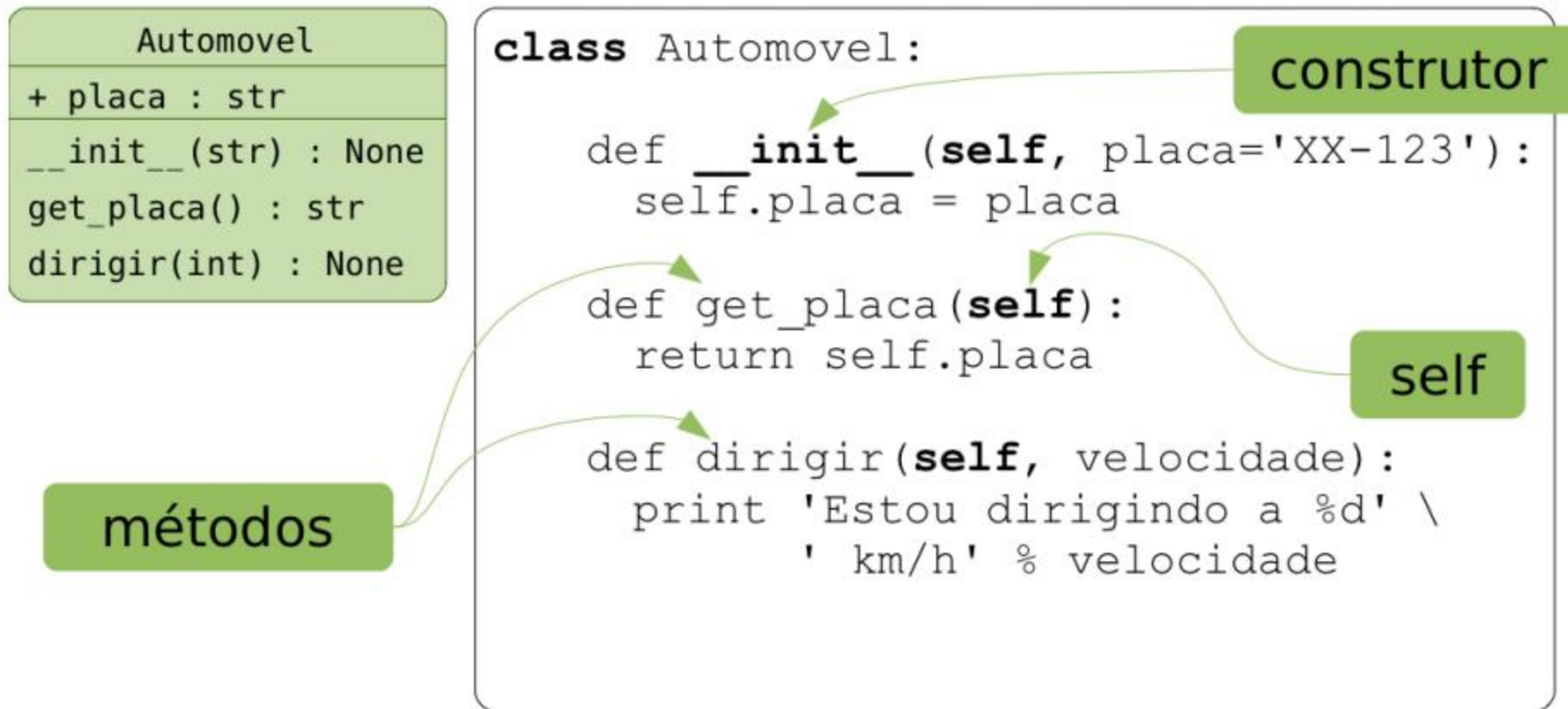
- Objetivo: entender classe, objeto, atributo, método e composição.



Orientação a Objetos: o básico com Aluno e Disciplina



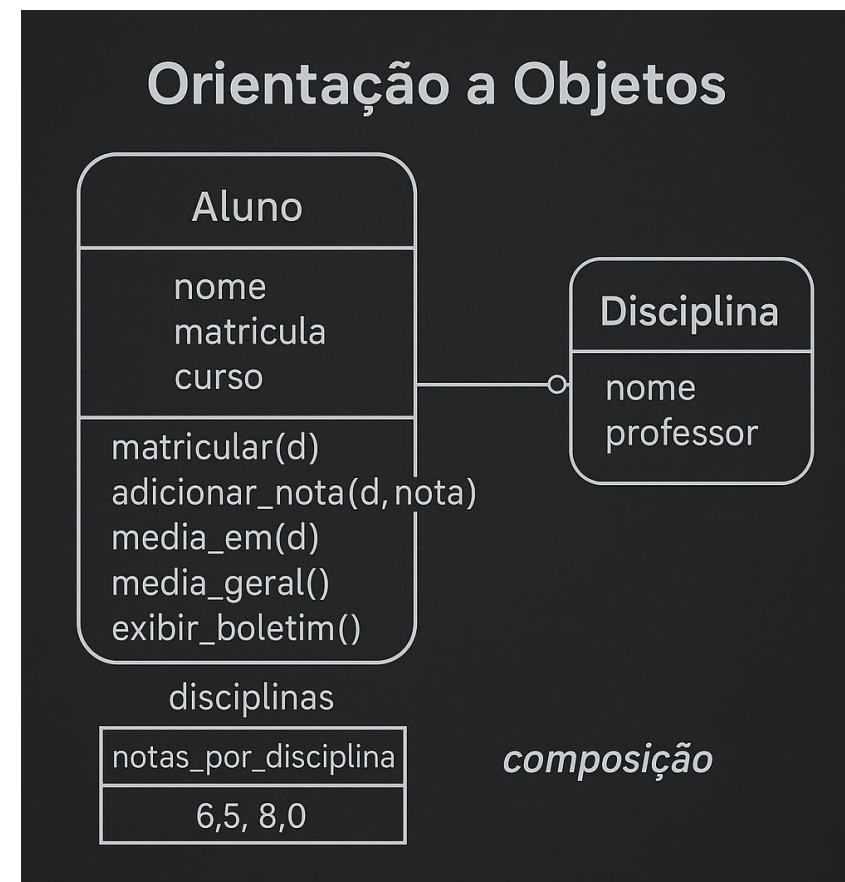
Orientação a Objetos: o básico com Aluno e Disciplina



Problema do mundo real

Queremos representar:

- Alunos (nome, matrícula, curso)
- Disciplinas (nome, professor)
- Notas pertencem ao aluno em cada disciplina



Conceitos-chave (visão rápida)

- **Classe:** molde (ex.: `Aluno`, `Disciplina`)
- **Objeto:** instância real (ex.: `aluno1`, `mat`)
- **Atributos:** dados do objeto (ex.: `nome`, `notas_por_disciplina`)
- **Métodos:** ações do objeto (ex.: `adicionar_nota`, `media_em`)
- **Composição:** um objeto contém outros (`Aluno` contém `Disciplinas`)

Disciplina: só dados da disciplina

```
class Disciplina: 7 usages new *
    def __init__(self, nome, professor): new *
        self.nome = nome
        self.professor = professor

    def exibir_informacoes(self): 1 usage (1 dynamic) new *
        print(f"Disciplina: {self.nome} | Professor: {self.professor}")
```

Pontos: sem notas aqui; disciplina não sabe desempenho do aluno.

Aluno: guarda notas por disciplina

```
from disciplina import Disciplina

class Aluno:
    def __init__(self, nome, matricula, curso):
        self.nome = nome
        self.matricula = matricula
        self.curso = curso
        self.disciplinas = []          # lista de objetos Disciplina
        self.notas_por_disciplina = {} # dict: nome_da_disciplina -> [notas]
```

Pontos: duas estruturas: lista de disciplinas + dicionário de notas.

Métodos do Aluno (matricular e notas)

```
def matricular(self, disciplina: Disciplina): 3 usages new *
    if disciplina not in self.disciplinas:
        self.disciplinas.append(disciplina)
    self.notas_por_disciplina.setdefault(disciplina.nome, [])

def adicionar_nota(self, disciplina: Disciplina, nota: float): 4 usages new *
    if disciplina.nome not in self.notas_por_disciplina:
        self.matricular(disciplina)
    self.notas_por_disciplina[disciplina.nome].append(nota)
```

Pontos: se não estiver matriculado, matricula; depois registra a nota.

Cálculo de médias (no Aluno)

```
def media_em(self, disciplina: Disciplina) -> float: 2 usages new *
    notas = self.notas_por_disciplina.get(disciplina.nome, [])
    if not notas:
        return 0.0
    return sum(notas) / len(notas)

def media_geral(self) -> float: 1 usage new *
    medias = []
    for disc in self.disciplinas:
        m = self.media_em(disc)
        if m > 0:
            medias.append(m)
    if not medias:
        return 0.0
    return sum(medias) / len(medias)
```

Pontos: média por disciplina e média geral só onde há notas.

Composição (desenho mental)

Aluno

- nome, matrícula, curso
- disciplinas: [Disciplina, Disciplina, ...]
- notas_por_disciplina: { "Matemática": [8.0, 6.5], "Física": [9.0] }

Disciplina

- nome
- professor

Pontos: Aluno **tem** Disciplinas; notas ficam no Aluno por disciplina.

App de demonstração

```
# app.py
from aluno import Aluno
from disciplina import Disciplina

# cria aluno
aluno1 = Aluno( nome: "João Silva", matricula: "2025001", curso: "Ciência da Computação")

# cria disciplinas
mat = Disciplina( nome: "Matemática", professor: "Prof. Carlos")
fis = Disciplina( nome: "Física", professor: "Prof. Ana")

# matricula aluno nas disciplinas
aluno1.matricular(mat)
aluno1.matricular(fis)

# adiciona **notas do aluno** em cada disciplina
aluno1.adicionar_nota(mat, nota: 8.0)
aluno1.adicionar_nota(mat, nota: 6.5)
aluno1.adicionar_nota(fis, nota: 9.0)
aluno1.adicionar_nota(fis, nota: 7.5)

# exibe boletim do aluno (com médias por disciplina e média geral)
aluno1.exibir_boletim()
```

Pontos: fluxo completo em poucas linhas.

App de demonstração

```
def exibir_boletim(self): 1 usage new *
    print(f"\nAluno: {self.nome} | Matrícula: {self.matricula} | Curso: {self.curso}")
    if not self.disciplinas:
        print("Sem disciplinas matriculadas.")
        return
    for d in self.disciplinas:
        notas = self.notas_por_disciplina.get(d.nome, [])
        media = self.media_em(d)
        d.exibir_informacoes()
        print(f"  Notas do aluno em {d.nome}: {notas if notas else '-'}")
        print(f"  Média do aluno em {d.nome}: {media:.1f}")
    print(f"MÉDIA GERAL: {self.media_geral():.1f}")
```

Pontos: cada disciplina mostra professor; aluno mostra notas e médias.

Mapa mental dos conceitos

- **Classe ↔ Objeto:** molde vs. instância.
- **Atributo:** estado do objeto.
- **Método:** comportamento do objeto.
- **Composição:** objetos dentro de objetos.

ALGUMA DÚVIDA?

Entre em contato pelo Teams ou por e-mail



<https://github.com/alexandrerussi>



Alexandre Russi Junior

Projects Coordinator | Higher Education Professor



[/alexandrerussi](#)